

Презентация по лабораторной работе №3

Компьютерный практикум по статистическому анализу

Арбатова Варвара Петровна

2026-09-21

Содержание I

- 1 Информация
- 2 Вводная часть
- 3 Задания и решения
- 4 Результаты
- 5 Обсуждение результатов
- 6 Заключение
- 7 Спасибо за внимание

Раздел 1

Информация

- Арбатова Варвара Петровна
- студентка
- Российский университет дружбы народов им. П. Лумумбы
- 1132236020@rudn.ru
- <https://v.p.arbatova.github.io/ru/>

Раздел 2

Вводная часть

- Julia сочетает читаемость Python с производительностью C.
- Управляющие конструкции (`while`, `for`, `if`) и функции высшего порядка (`map`, `broadcast`) — основа научных вычислений.
- Работа с матрицами и решение линейных систем — рутинные задачи исследователя.
- Лабораторная работа посвящена практическому освоению этих инструментов.

Объект и предмет исследования

- **Объект:** язык программирования Julia и его управляющие структуры.
- **Предмет:** применение циклов, функций и операций линейной алгебры к прикладным задачам.
- **Форматы:** Jupyter Notebook (`.ipynb`), отчёт в Markdown, презентация.

- **Цель:**

- Освоить управляющие структуры Julia и применить их к задачам линейной алгебры.

- **Задачи:**

- Решить 11 заданий лабораторной с использованием `while`, `for`, `if` и функций.
- Построить матрицы специальной структуры с помощью собственной функции `outer`.
- Решить систему линейных уравнений и проанализировать случайную матрицу.

- **Программное обеспечение:** Julia 1.12, Jupyter Notebook, пакет Random.
- **Методы:** написание и исполнение скриптов, сравнение map и broadcast, использование \ для решения линейных систем.
- **Воспроизводимость:** Random.seed! (42) фиксирует последовательность случайных чисел.

Раздел 3

Задания и решения

Задание 1. Циклы и коллекции

- Вывести целые числа от 1 до 100 и их квадраты.
- Построить словарь `squares (Int => Int)` с теми же данными.
- Построить массив `squares_arr` с помощью цикла `while`.

```
# Print integers 1..100 with their squares
for i in 1:100
  println("$i - ", i^2)
end
```

Задание 1. Результаты

- Словарь `squares`: **100** записей.
- `squares[1] = 1, squares[50] = 2500, squares[100] = 10000`.
- Массив `squares_arr: Vector{Int}` длины **100**.
- Оба цикла (`for` и `while`) дают одинаковый результат.

Задание 2. Условный и тернарный операторы

Условный оператор:

```
# Print the number if even, otherwise print "odd"
a = 4
if a % 2 == 0
    println(a)
else
    println("odd")
end
```

Тернарный оператор:

```
# Same logic using the ternary operator
x = 5
println(x % 2 == 0 ? x : "odd")
```

Задание 3. Функция add_one

```
# One-line function: add one to its argument  
add_one(x) = x + 1
```

- Однострочная форма объявления функции.
- Эквивалентна полной форме `function ... end`.

```
# Test the function  
println(add_one(5))      # 6  
println(add_one(0))      # 1  
println(add_one(-1))     # 0
```

Задание 4. map и broadcast

```
# Matrix A
A = [1 1 3; 5 2 6; -2 -1 -3]

# Three equivalent ways to add 1 to every element
map(add_one, A)      # using map
add_one.(A)          # using broadcast (dot syntax)
A .+ 1               # using broadcasting with a scalar
```

Результат (все три способа):

```
2  2  4
6  3  7
-1  0 -2
```

Задание 5. Матрица A , A^3 и замена столбца

$$A = \begin{pmatrix} 1 & 1 & 3 \\ 5 & 2 & 6 \\ -2 & -1 & -3 \end{pmatrix}$$

```
# Matrix power A^3 (not element-wise)
```

```
A3 = A3
```

```
# Replace the third column by the sum of the second and third
```

```
A_mod = copy(A)
```

```
A_mod[:, 3] = A[:, 2] .+ A[:, 3]
```

- A^3 — **матричное** возведение в степень.
- `copy(A)` — независимая копия.
- `A[:, 2]` — второй столбец как вектор.

Задание 5. Результаты

$$A^3 =$$

$$\begin{array}{rrr} -18 & -6 & -30 \\ -27 & -9 & -45 \\ 27 & 9 & 45 \end{array}$$

A   3- :

$$\begin{pmatrix} 1 & 1 & 4 \\ 5 & 2 & 8 \\ -2 & -1 & -4 \end{pmatrix}$$

Задание 6. Матрица B и $C = B^T B$

- B — матрица 15×3 со столбцами $[10, -10, 10]$.
- Столбцы взаимно ортогональны.

```
# Build matrix B: columns are 10, -10, 10
B = zeros(Int, 15, 3)
B[:, 1] .= 10
B[:, 2] .= -10
B[:, 3] .= 10

# Compute C = B' * B
C = B' * B
```

Результат:

1500	-1500	1500
-1500	1500	-1500
1500	-1500	1500

Задание 7. Матрицы Z_1 – Z_4 (6×6)

Четыре матрицы, заданные правилами по индексам:

Матрица	Правило
Z_1	$\text{abs}(i - j) \leq 1$
Z_2	$\text{abs}(i - j) \leq 1$ или $\text{abs}(i - j) == 5$
Z_3	$i + j \in \{5, 7, 9, 11\}$
Z_4	$\text{iseven}(i + j)$

```
# Build Z1: band of width 3 along the main diagonal
Z1 = zeros(Int, 6, 6)
for i in 1:6, j in 1:6
    abs(i - j) <= 1 && (Z1[i, j] = 1)
end
```

Задание 7. Результаты

Z1:						Z2:					Z3:					Z4:							
1	1	0	0	0	0	1	1	0	0	0	1	0	0	0	1	0	1	1	0	1	0	1	0
1	1	1	0	0	0	1	1	1	0	0	0	0	0	1	0	1	0	0	1	0	1	0	1
0	1	1	1	0	0	0	1	1	1	0	0	0	1	0	1	0	1	1	0	1	0	1	0
0	0	1	1	1	0	0	0	1	1	1	0	1	0	1	0	1	0	0	1	0	1	0	1
0	0	0	1	1	1	0	0	0	1	1	1	0	1	0	1	0	1	1	0	1	0	1	0
0	0	0	0	1	1	1	0	0	0	1	1	1	0	1	0	1	0	0	1	0	1	0	1

Задание 8. Собственная функция `outer(x, y, op)`

Аналог R-функции `outer()` с интерфейсом `outer(x, y, operation)`.

```
# Custom analog of R's outer() function
function outer(x, y, op)
  m, n = length(x), length(y)
  # Allocate an uninitialised matrix of the result type
  C = Matrix{typeof(op(x[1], y[1]))}(undef, m, n)
  for i in 1:m, j in 1:n
    C[i, j] = op(x[i], y[j])
  end
  return C
end
```

- `Matrix{T}(undef, m, n)` — неинициализированная матрица типа `T`.
- Тип `T` выводится из `op(x[1], y[1])`.
- Функция полностью обобщённая: работает с `+`, `*`, `^`, `mod` и др.

Задание 8. Матрицы A_1 – A_5

Матрица	Размер	Правило
A_1	5×5	$a + b$
A_2	5×5	$a * b^2$
A_3	5×5	$\text{mod}(a + b, 5)$
A_4	10×10	$\text{mod}(a + b, 10)$
A_5	9×9	$\text{mod}(a - b, 9)$

```
# Build matrices using the custom outer() function
A1 = outer(0:4, 0:4, (a, b) -> a + b)
A2 = outer(0:4, 0:4, (a, b) -> a * b^2)
A3 = outer(0:4, 0:4, (a, b) -> mod(a + b, 5))
A4 = outer(0:9, 0:9, (a, b) -> mod(a + b, 10))
A5 = outer(1:9, 1:9, (a, b) -> mod(a - b, 9))
```

Задание 9. Система линейных уравнений

$$\begin{cases} x_1 + 2x_2 + 3x_3 + 4x_4 + 5x_5 = 7 \\ 2x_1 + x_2 + 2x_3 + 3x_4 + 4x_5 = -1 \\ 3x_1 + 2x_2 + x_3 + 2x_4 + 3x_5 = -3 \\ 4x_1 + 3x_2 + 2x_3 + x_4 + 2x_5 = 5 \\ 5x_1 + 4x_2 + 3x_3 + 2x_4 + x_5 = 17 \end{cases}$$

Закономерность: $A[i, j] = |i - j| + 1$.

```
# Build matrix A using the pattern and solve the system
n = 5
A = [abs(i - j) + 1 for i in 1:n, j in 1:n]
b = [7, -1, -3, 5, 17]
x = A \ b
```

Задание 9. Решение

$$x = [1.0, -2.0, 3.0, -4.0, 5.0]$$

$$A^*x = [7.0, -1.0, -3.0, 5.0, 17.0]$$

$$b = [7, -1, -3, 5, 17]$$

- Вектор решения: $x = [1, -2, 3, -4, 5]$.
- Проверено подстановкой.
- Метод обобщается на любое n с той же структурой матрицы.

Задание 10. Случайная матрица M

- Матрица M размера 6×10 (в демонстрации — 6×6).
- Элементы: случайные целые из $1 \dots 10$ с повторениями.
- `Random.seed!(42)` для воспроизводимости.

```
# Generate a reproducible random matrix M
using Random
Random.seed!(42)
M = rand(1:10, 6, 6)
```

Задачи: 1. Число элементов $> N$ в каждой строке. 2. Строки, где `M_val` встречается ровно дважды. 3. Пары столбцов с суммой $> K$.

Задание 10. Результаты

Число > **4** в строке: [3, 3, 5, 3, 2, 3]

Строки, где 7 встречается дважды: [1, 5, 6]

Суммы столбцов: [34, 36, 19, 26, 32, 20]

Пары столбцов с суммой > **75**: нет (максимум — 70).

```
# Count elements > N in each row
count_per_row = [count(>(N), M[i, :]) for i in 1:6]

# Rows where M_val appears exactly twice
rows_with_two = [i for i in 1:6 if count(==(M_val), M[i, :]) == 2]

# Column sums and pairs with sum > K
col_sums = vec(sum(M, dims=1))
```

Задание 11. Двойные суммы

$$S_1 = \sum_{i=1}^{20} \sum_{j=1}^5 \frac{i^4}{3+j}, \quad S_2 = \sum_{i=1}^{20} \sum_{j=1}^5 \frac{i^4}{3+ij}.$$

```
# Double sum: denominator depends only on j
S1 = sum(i^4 / (3 + j) for i in 1:20, j in 1:5)

# Double sum: denominator depends on the product i*j
S2 = sum(i^4 / (3 + i*j) for i in 1:20, j in 1:5)
```

Результаты:

```
S1 = 639215.2833333338
S2 = 89912.02146097131
```

Раздел 4

Результаты

Сводная таблица

№	Задание	Ключевые конструкции Julia	Результат
1	Циклы и коллекции	for, while, Dict, push!	100 квадратов в словаре и векторе
2	Условия	if-else, ?:	Корректный вывод
3	add_one	Однострочная функция	6, 1, 0
4	map/broadcast	map, .(), .+	Одна и та же матрица тремя способами
5	A^3 , замена столбца	$\wedge$, copy, срез	A^3 , модифицированная A
6	$C = B \otimes B$	zeros, ', *	Структура ± 1500
7	$Z_1 - Z_4$	zeros, for, abs, mod	Четыре матрицы

Раздел 5

Обсуждение результатов

- **for vs while:** `for` более идиоматичен и безопасен; `while` удобен, когда число итераций заранее неизвестно.
- **Тернарный оператор:** краток, но не стоит вкладывать несколько условий — теряется читаемость.
- **map vs broadcast:** эквивалентны для одной коллекции; `broadcast` гибче для нескольких коллекций.
- **$A \setminus b$ vs $\text{inv}(A) * b$:** $\setminus$ быстрее и численно устойчивее.
- **outer:** демонстрирует обобщённое программирование — один и тот же код работает для чисел и булевых операций.

Раздел 6

Заключение

Управляющие структуры, функции высшего порядка и операции над матрицами в Julia позволяют решать прикладные задачи кратко, корректно и воспроизводимо — из единого источника Markdown/Julia.

Раздел 7

Спасибо за внимание